

Машинное обучение 1, ПМИ ФКН ВШЭ

Практическое домашнее задание 1

Общая информация

Дата выдачи: 25.01.2026

Мягкий дедлайн: 23:59MSK 08.02.2026

Жесткий дедлайн: 23:59MSK 14.02.2026

О задании

Задание состоит из двух разделов, посвященных работе с табличными данными с помощью библиотеки `polars` и визуализации с помощью `matplotlib`. В первом разделе вам предстоит выполнить базовые задания с помощью вышеуказанных библиотек, а во втором распределить студентов по курсам. Баллы даются за выполнение отдельных пунктов. Задачи в рамках одного раздела рекомендуется решать в том порядке, в котором они даны в задании.

Задание направлено на освоение jupyter notebook (будет использоваться в дальнейших заданиях), библиотек `polars` и `matplotlib`.

Оценивание и штрафы

Каждая из задач имеет определенную «стоимость» (указана в скобках около задачи). Максимально допустимая оценка за работу — 10 баллов без учёта бонусов. Сдавать задание после жёсткого дедлайна нельзя.

Задание выполняется самостоятельно. «Похожие» решения считаются плагиатом и все задействованные студенты (в том числе те, у кого списали) не могут получить за него больше 0 баллов (подробнее о плагиате см. на странице курса). Если вы нашли решение какого-то из заданий (или его часть) в открытом источнике, необходимо указать ссылку на этот источник (скорее всего вы будете не единственным, кто это нашел, поэтому чтобы исключить подозрение в плагиате, необходима ссылка на источник).

Устная проверка. Для проверки понимания кода и выводов студент может быть приглашён на устную защиту. Оценка за задание может быть изменена после устной защиты. Если студент не может объяснить ключевые части решения и принятые решения, работа считается недобросовестной и оценивается в 0 баллов независимо от автотестов.

Формат сдачи

Задания сдаются через систему Anytask. Инвайт можно получить у семинариста или ассистента. Присылать необходимо ноутбук с выполненным заданием. Сам ноутбук называйте в формате `homework-practice-01-polars-Username.ipynb`, где Username — ваша фамилия.

Для удобства проверки самостоятельно посчитайте свою максимальную оценку (исходя из набора решенных задач) и укажите ниже.

Оценка: $9.5 * 1.15 + 1$ (без циклов)

0. Введение

Сейчас мы находимся в jupyter-ноутбуке (или ipython-ноутбуке). Это удобная среда для написания кода, проведения экспериментов, изучения данных, построения визуализаций и других нужд, не связанных с написанием production-кода.

Ноутбук состоит из ячеек, каждая из которых может быть либо ячейкой с кодом, либо ячейкой с текстом размеченным и неразмеченным. Текст поддерживает markdown-разметку и формулы в Latex.

Для работы с содержимым ячейки используется *режим редактирования* (*Edit mode*, включается нажатием клавиши **Enter** после выбора ячейки), а для навигации между ячейками используется *командный режим* (*Command mode*, включается нажатием клавиши **Esc**). Тип ячейки можно задать в командном режиме либо с помощью горячих клавиш (**y** to code, **m** to markdown, **r** to edit raw text), либо в меню *Cell* -> *Cell type*.

После заполнения ячейки нужно нажать *Shift + Enter*, эта команда обработает содержимое ячейки: проинтерпретирует код или сверстает размеченный текст.

```
In [ ]: # ячейка с кодом, при выполнении которой появится output
        2 + 2
```

```
Out[ ]: 4
```

Ячейка с неразмеченным текстом.

Попробуйте создать свои ячейки, написать какой-нибудь код и текст какой-нибудь формулой.

```
In [ ]: # your code
        'bruh'
```

```
Out[ ]: 'bruh'
```

[Здесь](#) находится небольшая заметка о используемом языке разметки Markdown. Он позволяет:

0. Составлять упорядоченные списки
1. Выделять *текст* **при необходимости**

2. Добавлять [ссылки](#)

- Составлять неупорядоченные списки

Делать вставки с помощью LaTeX:

$$\begin{cases} x = 16 \sin^3(t) \\ y = 13 \cos(t) - 5 \cos(2t) - 2 \cos(3t) - \cos(4t) \\ t \in [0, 2\pi] \end{cases}$$

А ещё можно вставлять картинки, или гифки, или что захотите:



Google Colab

Что за колаб?

Google Colab (Colaboratory) это **Jupyter Notebook + Cloud + Google Drive**.

Компания Google предоставляет возможность бесплатно запускать ноутбуки (предварительно загрузив их на свой гугл-диск) прямо в облаке. При этом вам не требуется установка никаких пакетов на свою машину, а работать можно напрямую из браузера. Вот ссылка:

<https://colab.research.google.com>

При использовании вы увидите много сходств с jupyter ноутбуком. Одним из преимуществ является доступность GPU, соответствующую опцию можно активировать в настройках сервиса. При желании вы сможете найти в интернете много tutorиалов по использованию или разобраться самостоятельно =)

Kaggle

Альтернативой является **Kaggle**, он же площадка для самых разных соревнований по машинному обучению

Он, как и колаб, предоставляет бесплатный GPU, и даже не один, с чётким лимитом в 30 часов в неделю (colab может его внезапно отобрать, kaggle - нет) более мощные CPU с большим числом ядер, и больше Гб ОЗУ (раз так в 2.5)

<https://www.kaggle.com>

К сожалению мир не идеален, недостатки следующие:

- необходимость загружать датасеты через специальную кнопку
- невозможность использовать свои скрипты и модули :(
- нужно тыкать техподдержку, чтобы создать аккаунт из России

1. Табличные данные и Polars [4 балла]

Сперва повторим, почему `polars` такой клёвый, если вы не были на семинаре, либо не хотите с ним ознакомиться

Эта версия домашки предполагает, что вы уже являетесь уверенным пользователем `pandas` и знакомы с датафреймами, как они устроены и как ими манипулировать. Библиотека `polars` основана на тех же концептах.

Главные достоинства, которые чуть позже продемонстрируем:

- он написан на Rust, поэтому почти все операции в нём выполняются значительно быстрее, есть распараллеливание из коробки, автоматическая оптимизация запросов и пр
- он позволяет работать с данными, которые не помещаются в оперативку, с этим поможет `pl.LazyFrame` и параметр `engine / streaming`
- конечно же имеются дополнительные функции, которых в `pandas` нет, но очень хочется, например агрегации с расширяющимся окном, произвольные оконные функции и прочее, для этого есть крайне гибкая штука `pl.Expr`

Подробнее можете прочитать на [любом ресурсе, посвящённом polars](#)

В этой части потребуются выполнить несколько небольших заданий. Можно пойти двумя путями: сначала изучить материалы, а потом приступить к заданиям, или же разбираться "по ходу". Выбирайте сами

Если не понятно, как вызывать конкретный метод, обратитесь к материалам ниже. Имейте в виду, что синтаксис `polars` отличается от привычного нам пандасовского. Скорее всего все ваши самые извращённые фантазии уже как-то реализованы в `polars`, поэтому читайте внимательно. Ну и конечно гуглить не запрещено, но желательно прикладывать источник

Материалы:

1. [Getting started](#) из официального руководства
2. [Документация](#) со всем необходимым
3. [Модификация для Data Science](#), вам скорее всего не понадобится в данном задании, но имейте в виду

Многие из заданий можно выполнить несколькими способами. Не существуют единственно верного, но попробуйте максимально задействовать арсенал `polars` и

ориентируйтесь на простоту и понятность вашего кода. Мы не будем подсказывать, что нужно использовать для решения конкретной задачи, попробуйте находить необходимый функционал сами (название метода чаще всего очевидно)

```
In [1]: # поларс относительно молодой, имейте в виду, что что-то может сломаться, если б
!pip install -qU hvplot xlsx2csv polars
```

```
0.0/180.6 kB ? eta -:--:--
174.1/180.6 kB 10.1 MB/s eta 0:00:01
180.6/180.6 kB 3.5 MB/s eta 0:00:00
810.4/810.4 kB 13.2 MB/s eta 0:00:00
45.8/45.8 MB 16.6 MB/s eta 0:00:00
```

```
In [1]: import polars as pl
import gc
```

Скачаем данные:

```
In [3]: !wget -O 'end_seminar.xlsx' -q 'https://www.dropbox.com/sc/1/fi/2ezfw6b3y1umremf
```

Для пользователей Windows: скачайте файл самостоятельно и поместите его в папку с тетрадкой. Или попробуйте один из следующих вариантов:

```
In [4]: !powershell iwr -outf somefile https://somesite/somefile
```

/bin/bash: line 1: powershell: command not found

```
In [ ]: !pip install wget
import wget
wget.download('https://www.dropbox.com/sc/1/fi/2ezfw6b3y1umremf1bgdl/end_seminar.
```

Requirement already satisfied: wget in /usr/local/lib/python3.12/dist-packages (3.2)

```
Out[ ]: 'end_seminar.xlsx'
```

Обязательно прочитайте блок ниже

Пандасом и нампам пользоваться нельзя... Совсем, методами `to_pandas()` и `to_numpy()` тоже

В первой части задания (до раздела "Распределение студентов по курсам") использование циклов запрещается и повлечет за собой снижение оценки. Во второй уже можно, но с оговорками, о чём ниже

Циклами в данном случае считается `for`, `while` и `map`-методы `polars`, где можно использовать лямбда-функцию. Главным образом `map_elements` и его аналоги. И как правило, всё то, что не умеет делать `pl.LazyFrame` - список [здесь](#). Мотивация следующая - используя встроенные функции и методы, `polars` может оптимизировать ваш запрос и провести все нужные манипуляции, прежде чем сериализовать и вывести ответ (за некоторыми исключениями), поэтому важно научиться с этим управляться

Чтобы сделать выбор `polars` осмысленным, мы немножко модифицируем исходный датасет. Он будет больше в несколько раз и может не влезть в память, если у вас меньше 8 Гб, а с учётом преобразований может потребоваться до 64 Гб! Не пренебрегайте возможностями `pl.LazyFrame`. Его использование не обязательно, но может сильно облегчить вам жизнь

Трансформировать датасет `df` можно, как только это потребуется по условию (таких пунктов всего 2), на всякий случай поставим значок 

Если небрежно обращаться с методами `polars`, вы рискуете крашнуть ядро, взорвать компьютер, потерять рассудок... Если вдруг у вас есть тачка или хотите подебажить на исходном датасете, то пожалуйста, но помните про условия выше. В частности, при работе в колабе после ряда пунктов может остаться всякий мусор, не забывайте убирать за собой и пользоваться `gc.collect()`

Все, что не запрещено выше, разрешено

Гарантируется, что задание можно выполнить, имея лишь ресурсы Google Colab (бесплатные), удачи (◡‿◡)

Для каждой задачи из этого раздела вы должны написать код для получения ответа, а также дать текстовый ответ, если он предполагается.

На некоторые вопросы вы можете получить путём пристального взгляда на таблицу, но это не будет засчитываться. Вы в любом случае должны получить ответ с помощью кода.

Бонус polars. [1.5 балла]

Бонус за использование `polars` равен вашей оценке за дз, умноженной на коэффициент 1.15, не считая бонуса за отсутствие циклов, он прибавляется отдельно. Поэтому даже если сделали не всё, не отчаивайтесь!

Максимальная оценка за дз - $10 * 1.15$ (polars) + 1 (бонус без циклов)

0. Ещё пара красивых слов

Первые несколько манипуляций мы сделаем за вас, а заодно поясним, в чём тут нюанс

```
In [2]: # читаем датафрейм
initial_df = pl.read_excel("end_seminar.xlsx", engine="xlsx2csv")
print(initial_df.shape[0], "rows")
print(initial_df.estimated_size("mb"), "mb")
```

429 rows
0.18347549438476562 mb

Как видим, он достаточно маленький. С такими объёмами выбор библиотеки это чисто вкусовщина. Давайте немного поднимем ставки

```
In [3]: def repeat_df(df: pl.DataFrame | pl.LazyFrame, n: int = 2) -> pl.LazyFrame:
        return (
            df.lazy()
            .select(pl.col("timestamp").unique().sort().repeat_by(n).flatten())
            .join(df.lazy(), on="timestamp", how="left")
        )
```

Для работы с датафреймами, которые в память не влезают, нам понадобится объект `pl.LazyFrame`. Он представляет из себя схему выполнения вашего запроса, который оптимизируется, бьётся на батчи и параллелится под капотом через StreamingAPI, что для нас просто идеально. Перевести объект туда можно через метод `lazy`, обратно через `collect`. Учтите, что `streaming=True` работает не со всеми операциями, применяйте с умом, это можно проверить через `pl.LazyFrame.explain(streaming=True)`

```
In [4]: repeat_n = 2

repeated_df = repeat_df(initial_df, repeat_n)
print("Schema")
print(repeated_df.explain(streaming=True, optimized=True))
```

Schema

LEFT JOIN:

LEFT PLAN ON: [col("timestamp")]

SELECT [col("timestamp").unique().sort(asc).repeat_by([dyn int: 2]).explode()]
DF ["timestamp", "id", "rating", "group_22", ...]; PROJECT["timestamp"] 1/16

COLUMNS

RIGHT PLAN ON: [col("timestamp")]

DF ["timestamp", "id", "rating", "group_22", ...]; PROJECT["timestamp", "id",
"rating", "group_22", ...] 16/16 COLUMNS

END LEFT JOIN

C:\Users\8dant\AppData\Local\Temp\ipykernel_7628\2548353322.py:4: DeprecationWarning: `Expr.flatten()` is deprecated and will be removed in version 2.0. Use `Expr.list.explode(keep\_nulls=False, empty\_as\_null=False)` instead.

.select(pl.col("timestamp").unique().sort().repeat_by(n).flatten())

C:\Users\8dant\AppData\Local\Temp\ipykernel_7628\2973500786.py:5: DeprecationWarning: the `streaming` parameter was deprecated in 1.25.0; use `engine` instead.

print(repeated_df.explain(streaming=True, optimized=True))

```
In [5]: # как видим, повторы работают
repeated_df = repeated_df.collect()
assert repeated_df.shape[0] == initial_df.shape[0] * repeat_n
print(repeated_df.shape[0], "rows")
print(repeated_df.estimated_size("mb"), "mb")
```

858 rows

0.3668785095214844 mb

Теперь ещё интереснее. Если бы наш датасет был не в 2 раза больше, а в 100000 раз, что бы мы делали? К счастью для нас, поларс может уместить 300кк строк всего лишь в 512Мб, и это даже не прикол, но работать с этим придётся немного иначе

```
In [6]: N = 80000

df = repeat_df(initial_df, N)
print(initial_df.estimated_size("gb") * N, "gb")
```

14.334022998809814 gb

C:\Users\8dant\AppData\Local\Temp\ipykernel_7628\2548353322.py:4: DeprecationWarning: `Expr.flatten()` is deprecated and will be removed in version 2.0. Use `Expr.list.explode(keep\_nulls=False, empty\_as\_null=False)` instead.
.select(pl.col("timestamp").unique().sort().repeat_by(n).flatten())

Поэтому даже такая тривиальная задача, как подсчёт строк становится уже не такой простой. Про это и будет дз

```
In [7]: try:
        print(df.shape[0], "rows")
except AttributeError:
    print("ой, не получилось...")

# df.collect().shape[0] # тоже можно, если влезет
# len(df.collect().to_pandas()) # а такое может крашнуть ядро, попробуйте, хехе.
```

ой, не получилось...

```
In [8]: def lazy_len(df: pl.LazyFrame) -> int:
        return df.select(pl.len()).collect().item()

rows = lazy_len(df) # намного экономнее, потому что оптимизировано под капотом
assert rows == initial_df.shape[0] * N
print(rows, "rows")
```

34320000 rows

Далее нужно работать с увеличенным датафреймом `df`

1. [0.5 баллов] Выведите 3 последних строки

Посмотрите на данные и скажите, что они из себя представляют, сколько в таблице строк, какие столбцы? (на это не надо отвечать, просто подумайте об этом)

```
In [9]: df.tail(3).collect()
```


Out[9]: shape: (3, 16)

timestamp	id	rating	group_22	is_mi	fall_1
str	str	i64	i64	i64	str
"2024-08-23 16:40:22"	"f5de8b91d28d8d19bf721b14fce501..."	1504	null	null	"Операционные системы 2"
"2024-08-23 16:40:22"	"f5de8b91d28d8d19bf721b14fce501..."	1504	null	null	"Операционные системы 2"
"2024-08-23 16:40:22"	"f5de8b91d28d8d19bf721b14fce501..."	1504	null	null	"Операционные системы 2"

Ответьте на вопросы:

1. Сколько было уникальных пользователей из групп 22-го года набора, а сколько из групп 21-го года?

```
In [10]: answer22 = lazy_len(df.filter(pl.col('group_22').is_not_null() & pl.col('id').is
print(answer22)
print('-' * 50)
answer21 = lazy_len(df.filter(pl.col('group_21').is_not_null() & pl.col('id').is
print(answer21)
```

259

159

2. Есть ли среди объединенного множества 3-го и 4-го курса уникальные студенты с равными перцентилями?

```
In [11]: res = (df.unique(subset='id')
            .group_by(pl.col('percentile'))
            .agg(pl.col('id').count().alias('counter'))
            .filter(pl.col('counter') > 1)
            )
print(f'Кол-во уникальных пользователей с одинаковым перцентилем: {res.select(pl

# df.unique(subset='id').filter(pl.col('percentile').is_in(res.collect()['percen
```

Кол-во уникальных пользователей с одинаковым перцентилем: 160

2. [0.5 балла] Есть ли в данных пропуски? В каких колонках? Сколько их в каждой из этих колонок?

Уникальных или нет - не столь важно

```
In [12]: df.null_count().collect(streaming=True) # если не ноль, то значит есть. По сути
```

```
C:\Users\8dant\AppData\Local\Temp\ipykernel_7628\1256127211.py:1: DeprecationWarning: the `streaming` parameter was deprecated in 1.25.0; use `engine` instead.  
df.null_count().collect(streaming=True) # если не ноль, то значит есть. По сути  
пропуски в таком кол-ве из-за замножения данных в самом начале
```

```
Out[12]: shape: (1, 16)
```

timestamp	id	rating	group_22	is_mi	fall_1	fall_2	fall_3	spring_1	spring_2	spring_3
u32	u32	u32	u32	u32	u32	u32	u32	u32	u32	u32
0	0	0	12720000	32240000	0	0	0	3840000	3840000	3840000



 Преобразуйте датасет - заполните пропуски пустой строкой для строковых колонок, нулём для числовых и False для булевых (постарайтесь избежать перечисления названий всех столбцов). Опирайтесь не на смысл столбцов, а на текущий тип данных.

```
In [13]: df.schema
```

```
C:\Users\8dant\AppData\Local\Temp\ipykernel_7628\3182147272.py:1: PerformanceWarning: Resolving the schema of a LazyFrame is a potentially expensive operation. Use `LazyFrame.collect_schema()` to get the schema without this warning.  
df.schema
```

```
Out[13]: Schema([('timestamp', String),  
                ('id', String),  
                ('rating', Int64),  
                ('group_22', Int64),  
                ('is_mi', Int64),  
                ('fall_1', String),  
                ('fall_2', String),  
                ('fall_3', String),  
                ('spring_1', String),  
                ('spring_2', String),  
                ('spring_3', String),  
                ('is_first_time', String),  
                ('percentile', Float64),  
                ('group_21', String),  
                ('blended', String),  
                ('is_ml_student', Boolean)])
```

```
In [14]: df_modified = (  
    df  
    .with_columns(pl.col(pl.String).fill_null(''))  
    .with_columns(pl.col(pl.Float64), pl.col(pl.Int64)).fill_null(0)  
    .with_columns(pl.col(pl.Boolean)).fill_null(False)  
    )  
print(lazy_len(df_modified))  
  
df_modified.null_count().collect(streaming=True)
```

34320000

```
C:\Users\8dant\AppData\Local\Temp\ipykernel_7628\2603033888.py:9: DeprecationWarning: the `streaming` parameter was deprecated in 1.25.0; use `engine` instead.  
df_modified.null_count().collect(streaming=True)
```

Out[14]: shape: (1, 16)

timestamp	id	rating	group_22	is_mi	fall_1	fall_2	fall_3	spring_1	spring_2	spring_3
u32	u32	u32	u32	u32	u32	u32	u32	u32	u32	u32
0	0	0	0	0	0	0	0	0	0	0



3. [0.5 балла] Посмотрите повнимательнее на колонку 'is_first_time'.

Каково процентное соотношение ответов? Сколько из них "Нет"?

```
In [15]: result = (  
    df_modified.select(  
        pl.len().alias('total'),  
        pl.col('is_first_time')  
            .str.strip_chars() # просто на всякий случай сделал. Там вероятно зап  
            .str.to_lowercase() # просто на всякий случай сделал. Там вероятно зап  
            .eq('нет')  
            .sum()  
            .alias('counter')  
    )  
    .collect()  
)  
  
total = result['total'][0]  
f = result['counter'][0]  
  
print(f'Процент Нет: {f / total * 100}')  
print(f'Процент Да: {(total - f) / total * 100}') # там только Да и Нет, поэтому  
print(f'Кол-во Нет: {f}') # такое кол-во из-за замножения, но мы в одинаковое ко
```

Процент Нет: 5.128205128205128

Процент Да: 94.87179487179486

Кол-во Нет: 1760000

Посмотрите на колонку `timestamp` - верно ли, что датафрейм отсортирован по ней?

```
In [16]: checker = (  
    df_modified  
        .select('timestamp')  
        .with_columns(  
            pl.col('timestamp').shift(1).alias('previous')  
        )  
        .filter(  
            pl.col('timestamp') < pl.col('previous')  
        )  
        .limit(100)  
    )  
  
checker.collect(streaming=True)
```

```
C:\Users\8dant\AppData\Local\Temp\ipykernel_7628\520938460.py:13: DeprecationWarning: the `streaming` parameter was deprecated in 1.25.0; use `engine` instead.  
checker.collect(streaming=True)
```

Out[16]: shape: (100, 2)

timestamp	previous
str	str
"2024-08-13 18:06:52"	"2024-08-13 19:26:10"
"2024-08-11 21:45:25"	"2024-08-15 01:29:14"
"2024-08-11 21:40:11"	"2024-08-11 21:45:25"
"2024-08-15 00:50:51"	"2024-08-15 01:29:14"
"2024-08-11 21:58:35"	"2024-08-15 00:50:51"
...	...
"2024-08-14 16:05:01"	"2024-08-14 17:21:36"
"2024-08-14 14:50:57"	"2024-08-14 18:24:52"
"2024-08-13 20:58:16"	"2024-08-14 15:18:50"
"2024-08-11 22:14:10"	"2024-08-13 21:16:52"
"2024-08-14 21:01:10"	"2024-08-14 21:28:57"

df не сортировали по дате

 Оставьте только самую позднюю версию обращений студентов, уберите колонку `is_mi` (она нам не пригодится в дальнейшем) и сохраните изменения. Дубли оставлять тоже не нужно

Обращения со значением "Нет" в 'is_first_time' могут быть как повторными, так и первичными, поскольку поле заполняли сами студенты.

```
In [17]: print(lazy_len(df_modified))  
  
df_modified = (  
    df_modified  
    .group_by('id')  
    .agg(pl.all().sort_by('timestamp', descending=False).last())  
)  
  
if 'is_mi' in df_modified.columns:  
    df_modified = df_modified.drop('is_mi')  
  
print(lazy_len(df_modified))  
  
df = df_modified.collect(streaming=True)
```

34320000

```
C:\Users\8dant\AppData\Local\Temp\ipykernel_7628\4188002203.py:9: PerformanceWarning: Determining the column names of a LazyFrame requires resolving its schema, which is a potentially expensive operation. Use `LazyFrame.collect_schema().names()` to get the column names without this warning.  
    if 'is_mi' in df_modified.columns:
```

418

```
C:\Users\8dant\AppData\Local\Temp\ipykernel_7628\4188002203.py:14: DeprecationWarning: the `streaming` parameter was deprecated in 1.25.0; use `engine` instead.  
    df = df_modified.collect(streaming=True)
```

```
In [18]: df.filter(pl.col('id') == 'afcd52c59f1bfedcd5415fbdf19cb410e68de8a37687d8f5e98b1')
```

Out[18]: shape: (1, 15)

	id	timestamp	rating	group_22	fall_1	fall_2
	str	str	i64	i64	str	str
	"afcd52c59f1bfedcd5415fbdf19cb4...	"2024-08-18 15:06:13"	630	226	"Системы баз данных"	"Statistical Learning Theory"

```
In [19]: df
```

Out[19]: shape: (418, 15)

	id	timestamp	rating	group_22	fall_1
	str	str	i64	i64	str
	"b8ab1a4ad4926caca7b82f8d1d11b2..."	"2024-08-11 23:57:32"	1158	0	"Дизайн систем"
	"40c3044ac2c56f3b576601f7052685..."	"2024-08-14 17:20:09"	755	225	"Разработка микросервисов на Go" пр
	"15a368d1368e4fd9b6dc0f2bc9362f..."	"2024-08-14 00:35:41"	607	222	"Основы разработки компьютерных..."
	"e6b3b61c7e6a0dcdd69e96325d12b2..."	"2024-08-13 19:31:46"	680	229	"Основы разработки компьютерных..." в
	"274f10a994f43939375bdfb3b3ac23..."	"2024-08-20 11:28:32"	696	229	"Прикладная статистика в машинн..." н
	...	...	...	...	...
	"3501992c6601498ed885bfb2deefe..."	"2024-08-14 13:59:21"	933	0	"Байесовские методы в машинном ..." в
	"48fa126659b29ec4bfcd13cbc83506..."	"2024-08-14 22:43:39"	1285	0	"Дизайн систем"
	"edb64996c64421378537d941a77426..."	"2024-08-14 23:57:53"	1086	0	"Операционные системы 2" в
	"d41f00051219658f10ba5bd41aba05..."	"2024-08-14 22:53:28"	634	226	"Распределенные системы"
	"8edbf92455432dd49c7966b801be12..."	"2024-08-11 23:53:53"	717	225	"Распределенные системы"

4. [0.5 балла] Какие blended-курсы для четверокурсников существуют? На какой blended-курс записалось наибольшее количество студентов? На каком из blended-курсов наименьший средний перцентиль студентов, подавших заявку (выведите этот курс и количество студентов на нем)?

In [20]: `df.filter(pl.col('blended') != '').select(pl.col('blended')).unique()`

Out[20]: shape: (6, 1)

blended
str
"Протоколы доказательств с нуле...
"ML System Design"
"Продуктовый подход к анализу д...
"Безопасность систем на базе LL...
"Введение в дифференциальную ге...
"Соревновательный анализ данных"

```
In [21]: stata = (df
            .filter(pl.col('blended') != '')
            .group_by(pl.col('blended'))
            .agg(
                unique=pl.col('id').n_unique(),
                total=pl.col('id').count(),
                avg=pl.col('percentile').mean()
            )
            .sort(pl.col('total'), descending=True)
        )

stata.limit(1)
```

Out[21]: shape: (1, 4)

blended	unique	total	avg
str	u32	u32	f64
"Протоколы доказательств с нуле...	44	44	0.56325

```
In [22]: stata.sort(pl.col('avg'), descending=False).limit(1)
```

Out[22]: shape: (1, 4)

blended	unique	total	avg
str	u32	u32	f64
"Продуктовый подход к анализу д...	25	25	0.39195

5. [1 балл] Выясните, есть ли студенты с абсолютно одинаковыми предпочтениями по всем курсам.

Для этого сформируйте таблицу, где для каждого возможного набора курсов посчитано количество студентов, выбравших такой набор, и оставьте только строки где это количество больше 1. Формировать варианты, которые не представлены ни одним студентом изначально, не требуется.

В данном случае набор курсов задается упорядоченным множеством ('fall_1', 'fall_2', 'fall_3', 'spring_1', 'spring_2', 'spring_3', 'blended'). Элемент blended будет нулевым для

3-го курса.

```
In [23]: nabori = (df
            .group_by(pl.col('fall_1'), pl.col('fall_2'), pl.col('fall_3'), pl.col('
            .agg(
                unique=pl.col('id').n_unique(),
                total=pl.col('id').count()
            )
            .filter(pl.col('total') > 1)
        )
        nabori
```

Out[23]: shape: (15, 9)

fall_1	fall_2	fall_3	spring_1	
str	str	str	str	
"Системы баз данных"	"Операционные системы 2"	"Разработка микросервисов на Go"	"Компьютерные сети"	"Компь
"Системы баз данных"	"Системы баз данных"	"Системы баз данных"	"Компьютерные сети"	"Компь
"Безопасность компьютерных сист..."	"Основы разработки компьютерных..."	"Язык SQL"	"Компьютерные сети"	"Компь
"Системы баз данных"	"Системы баз данных"	"Системы баз данных"	"Компьютерные сети"	программи
"Системы баз данных"	"Операционные системы 2"	"Функциональное программировани..."	"..."	
...	...	...	...	
"Системы баз данных"	"Операционные системы 2"	"Разработка микросервисов на Go"	"..."	
"Дизайн систем"	"Безопасность компьютерных сист..."	"Основы разработки компьютерных..."	"Децентрализованные системы"	"Промы программи
"Дизайн систем"	"Безопасность компьютерных сист..."	"Основы разработки компьютерных..."	"Децентрализованные системы"	"Методы передачи
"Системы баз данных"	"Распределенные системы"	"Разработка микросервисов на Go"	"..."	
"Системы баз данных"	"Разработка микросервисов на Go"	"Основы разработки компьютерных..."	"Язык программирования Go"	"Компь



6. [0.5 балла] Найдите курсы по выбору, на которые записывались как студенты 22-го года набора, так и студенты 21-го года.

```
In [24]: group21 = (
    df
    .filter(pl.col('group_21') != '')
    .select('fall_1', 'fall_2', 'fall_3', 'spring_1', 'spring_2', 'spring_3', 'b
    .melt()
    .filter(pl.col('value') != '')
    .select(pl.col('value').alias('course'))
    .unique()
)

group22 = (
    df
    .filter(pl.col('group_22') != 0)
    .select('fall_1', 'fall_2', 'fall_3', 'spring_1', 'spring_2', 'spring_3', 'b
    .melt()
    .filter(pl.col('value') != '')
    .select(pl.col('value').alias('course'))
    .unique()
)

intersection = group21.join(
    group22,
    on='course',
    how='inner'
)

intersection
```

```
C:\Users\8dant\AppData\Local\Temp\ipykernel_7628\2875483942.py:5: DeprecationWarning: `DataFrame.melt` is deprecated; use `DataFrame.unpivot` instead, with `index` instead of `id_vars` and `on` instead of `value_vars`
    .melt()
```

```
C:\Users\8dant\AppData\Local\Temp\ipykernel_7628\2875483942.py:15: DeprecationWarning: `DataFrame.melt` is deprecated; use `DataFrame.unpivot` instead, with `index` instead of `id_vars` and `on` instead of `value_vars`
    .melt()
```

Out[24]: shape: (14, 1)

course
str
"Типы в языках программирования"
"Основы информационного поиска"
"Основы тензорных вычислений"
"Безопасность компьютерных сист..."
"Специальные разделы матричного..."
...
"Математические основы нейросет..."
"Операционные системы 2"
"Стохастический анализ"
"Statistical Learning Theory"
"Язык программирования Scala"

Методом исключения найдите курсы, которые предлагались только студентам 22-го года и только студентам 21-го года.

```
In [25]: only21 = group21.join(  
    intersection,  
    on='course',  
    how='anti'  
)  
only21
```

Out[25]: shape: (29, 1)

course
str
"Глубинное обучение в анализе г..."
"Конфликты и кооперация (препод..."
"Трёхмерное компьютерное зрение"
"Дизайн систем"
"Генеративные модели в машинном..."
...
"Теория и практика онлайн-экспе..."
"Развёртывание ML-моделей в выс..."
"Соревновательный анализ данных"
"Протоколы доказательств с нуле..."
"Байесовские методы в машинном ..."

```
In [26]: only22 = group22.join(  
    intersection,  
    on='course',  
    how='anti'  
)  
only22
```

Out[26]: shape: (24, 1)

course
str
"Рекомендательные системы"
"Компьютерные сети"
"ШАД RL"
"Язык SQL"
"Количественные финансы"
...
"Лингвистика для программистов"
"Язык программирования Go"
"Генеративные модели в машинном..."
"Аналитика данных"
"Разработка микросервисов на Go"

Видите ли вы что-то необычное в получившихся списках? (менять ничего не нужно, просто подумайте)

Ответ:

Визуализации

При работе с данными часто неудобно делать какие-то выводы, если смотреть на таблицу и числа в частности, поэтому важно уметь визуализировать данные. Здесь будут описаны ключевые правила оформления графиков для **всех** домашних заданий.

У `matplotlib`, конечно же, есть [документация](#) с большим количеством [примеров](#), но для начала достаточно знать про несколько основных типов графиков:

- `plot` — обычный поточечный график, которым можно изображать кривые или отдельные точки;
- `hist` — гистограмма, показывающая распределение некоторой величины;
- `scatter` — график, показывающий взаимосвязь двух величин;
- `bar` — столбцовый график, показывающий взаимосвязь количественной величины от категориальной.

В этом задании вы попробуете построить один из них. Не забывайте про базовые принципы построения приличных графиков:

- оси должны быть подписаны, причём не слишком мелко;
- у графика должно быть название;
- если изображено несколько графиков, то необходима поясняющая легенда;
- все линии на графиках должны быть чётко видны (нет похожих цветов или цветов, сливающихся с фоном);
- если возможно сделать график интерактивным, это обычно того стоит - помогает решить проблемы с читаемостью;
- если отображена величина, имеющая очевидный диапазон значений (например, проценты могут быть от 0 до 100), то желательно масштабировать ось на весь диапазон значений (исключением является случай, когда вам необходимо показать малое отличие, которое незаметно в таких масштабах);
- сетка на графике помогает оценить значения в точках на глаз, это обычно полезно, поэтому лучше ее отрисовывать;
- если распределение на гистограмме имеет тяжёлые хвосты, лучше использовать логарифмическую шкалу.

Еще одна библиотека для визуализации: [seaborn](#). Это настройка над `matplotlib`, иногда удобнее и красивее делать визуализации через неё.

У `polars` уже есть встроенные инструменты для визуализации, в частности интерактивные на базе [hvplot](#) и [altair](#). Из её преимуществ - хорошая производительность при работе с большими данными, есть поддержка самых популярных видов графиков, но синтаксис не самый удобный

7. [0.5 балла] Постройте график (barplot) медианных перцентилей в зависимости от первого символа hex-кода id студента

Таким образом мы на глаз прикидываем, есть ли зависимость между, якобы, случайным id и значимым параметром. Если зависимость есть, и в дальнейшем мы будем сплитовать данные на тест и трейн по такому айдишнику без перемешивания, то можем допустить утечку, поэтому такие особенности важно отслеживать.

В графике явно задайте порядок столбцов по символам hex-алфавита (от 0 до f слева направо) и настройте отображение доверительных интервалов уровня **0.96**. Можно ли говорить о наличии проверяемой зависимости на основе визуальной информации?

In []: `# your code here`

Сохраните график в формате pdf (так он останется векторизованным) или html (тогда останется интерактивным)

In []: `# your code here`

2. Распределение студентов по курсам. (6 баллов + 1 бонус)

!!ВНИМАТЕЛЬНО ИЗУЧИТЕ ТЕКСТ НИЖЕ!!.

Если во время выполнения заданий у вас возникнут вопросы - еще раз перечитайте текст целиком, скорее всего ответы уже содержатся в нем.

Теперь вам нужно распределить студентов по осенним курсам по выбору, учитывая их предпочтения.

Алгоритм распределения студентов по курсам:

1. Для каждой дисциплины формируется список тех, кто указал её первым приоритетом (если студент должен выбрать два курса по выбору, то для него дисциплины, которые он указал первым и вторым приоритетом, рассматриваются как дисциплины первого приоритета). Если желающих больше, чем мест (см ниже), то выбирается топ по перцентилью рейтинга.
2. На дисциплинах, где остались места после первой волны, формируются списки тех, кто выбрал их вторым приоритетом, и места заполняются лучшими по перцентилью рейтинга студентами. После этого проводится такая же процедура для дисциплин третьего приоритета.
3. Если студент не попал на необходимое количество курсов по итогам трёх волн, с ним связывается учебный офис и решает вопрос в индивидуальном порядке.

Обращаем ваше внимание на следующие детали:

Особенности реализации алгоритма:

- Конкурс на каждый курс общий для 3-го (22-й год) и 4-го курса (21-й год), то есть список формируется из студентов обоих курсов.
- По умолчанию на каждой дисциплине по выбору у 3 и 4 курсов может учиться 1 группа (до 30 студентов).
 - Исключения:
 - Количественные финансы - 60
 - Промышленное программирование на Haskell - 60
 - Рекомендательные системы - 60
 - Глубинное обучение в обработке звука - 1000 (да да, не удивляйтесь! раньше и не такое было...)
- По умолчанию студент выбирает один осенний и один весенний курс по выбору, а также четверокурсники выбирают один blended-курс. Студенты групп **21-го года специализации МОП** выбирают по 2 осенних и 2 весенних курса по выбору. *Для студентов, которые выбирают 2 курса (например, осенних) первый приоритет — `faLL_1` и `faLL_2`, второй приоритет — `faLL_3`. Такие студенты участвуют только в двух волнах отбора.*
- Студенты специализации МОП не могут выбрать весенним курсом по выбору Машинное обучение 2. Если студент специализации МОП выбрал Машинное обучение 2, то его приоритеты сдвигаются.
 - "Сдвиг" = приоритеты 2 и 3 становятся приоритетами 1 и 2, 3-й приоритет остается пустым.
 - Из-за совпадений первого и второго курса по выбору в прочих случаях двигать приоритеты не надо.
- Blended-курсы не трогайте, по ним не надо распределять, на другие курсы они никак не влияют.
- В рамках своего решения вы можете считать, что в процессе распределения не возникнет ситуации, когда на одно место претендуют студенты с одинаковым перцентилем.

Я МОП и я люблю Я

Принадлежность к стойлу специализации МОП определяется так:

- 21-й год (4-й курс):
 - 211, 212, 213 - МОП; остальные - не МОП! 
- 22-й год (3-й курс)

- Студенты, у которых проставлен флаг в колонке `is_ml_student`.

Рейтинг и перцентиль

Рейтинг

В столбце `'rating'` записана кредитно-рейтинговая сумма студента (грубо говоря, сумма оценок студента по всем его дисциплинам с весами (веса > 1) — чем дольше шла дисциплина, тем больше вес; подробности тут:

<https://www.hse.ru/studyspravka/rate/>).

Чем больше курсов взял студент за период обучения, и чем лучше он их закрыл, тем выше его значение `'rating'`, соответственно, эта метрика возрастает по "качеству" студента.

Перцентиль

В столбце `'percentile'` записан перцентиль студента в рамках своего курса своей ОП.

Перцентиль студента X определяется следующим образом: "процент студентов, чьи результаты обучения лучше, чем у студента X". Соответственно, 0% -- лучший студент на потоке, → 100% -- худший студент на потоке, то есть метрика убывает по "качеству" студента.

Мы распределяем студентов по топу перцентилей, то есть по наименьшему перцентиле.

За прочими подробностями отправляем вас в ноутбук семинара.

Требования к решению

- Постарайтесь воздержаться от использования циклов там, где это возможно. Допустимо итерироваться по **курсам**, на которые проводится отбор, и по **волнам** отбора. *Если вы придумаете, как обойтись без цикла по **курсам**, то можете получить +1 бонусный балл к итоговой оценке. **Дублирование кода не признается успешным избавлением от циклов***
- На выходе ожидается файл `res_fall.csv` с результатами распределения на осенние курсы по выбору. Файл должен быть следующего формата:
 - три столбца: ID, course1, course2
 - Если студент не попал на курс, но должен был, то вместо названия курса в ячейке должна быть строка "???"
 - Если студент должен выбрать только один курс, то в колонке course2 для него должна стоять строка "-"

- Если студент должен выбрать два курса по выбору, то порядок в колонках course1 и course2 не важен.
- Формат csv: для сохранения воспользуйтесь `df.to_csv('solution.csv', index=None)`

Для работы вам могут понадобиться следующие данные:

- Результаты опроса (вы уже использовали этот файл в первой части задания, но на всякий случай ссылка:

https://www.dropbox.com/scl/fi/2ezfw6b3y1umremflbgdl/end_seminar.xlsx?rlkey=xz ecol82ybnpu0eon0b1s11pp&st=f45sugvp&dl=0

Кстати, убедитесь, что в данных больше нет пропусков, повторных записей и лишних колонок

0. Проверка

Для начала давайте убедимся, что вы успешно выполнили задания первой части и проверим ваши данные на наличие пропусков и повторов

```
In [27]: assert type(df) != pl.LazyFrame and type(df) == pl.DataFrame, "Датафрейм не собран"
assert df.shape == (418, 15), "В таблице остались повторы или потеряны данные"
assert df.null_count().sum_horizontal().item() == 0, "В таблице остались пропуски"
```

Если вы не получили `AssertionError`, то можете продолжать.

1. [1 балл] Создайте новый признак, обозначающий, сколько осенних курсов должен выбрать студент

В этом вам может помочь информация о специализации и группе студента.

```
In [28]: df = df.with_columns(
    pl.when(
        (pl.col('group_21').is_in(['211', '212', '213'])))
    .then(2)
    .otherwise(1)
    .alias('courses_count')
)
df
```


Out[28]: shape: (418, 16)

	id	timestamp	rating	group_22	fall_1
	str	str	i64	i64	str
	"b8ab1a4ad4926caca7b82f8d1d11b2..."	"2024-08-11 23:57:32"	1158	0	"Дизайн систем"
	"40c3044ac2c56f3b576601f7052685..."	"2024-08-14 17:20:09"	755	225	"Разработка микросервисов на Go" пр
	"15a368d1368e4fd9b6dc0f2bc9362f..."	"2024-08-14 00:35:41"	607	222	"Основы разработки компьютерных..."
	"e6b3b61c7e6a0dcdd69e96325d12b2..."	"2024-08-13 19:31:46"	680	229	"Основы разработки компьютерных..." в
	"274f10a994f43939375bdfb3b3ac23..."	"2024-08-20 11:28:32"	696	229	"Прикладная статистика в машинн..." н
	...	...	...	...	...
	"3501992c6601498ed885bfbb2deefe..."	"2024-08-14 13:59:21"	933	0	"Байесовские методы в машинном ..." в
	"48fa126659b29ec4bfcd13cbc83506..."	"2024-08-14 22:43:39"	1285	0	"Дизайн систем"
	"edb64996c64421378537d941a77426..."	"2024-08-14 23:57:53"	1086	0	"Операционные системы 2" в
	"d41f00051219658f10ba5bd41aba05..."	"2024-08-14 22:53:28"	634	226	"Распределенные системы"
	"8edbf92455432dd49c7966b801be12..."	"2024-08-11 23:53:53"	717	225	"Распределенные системы"

Проверка:

```
In [29]: col_name = 'courses_count'

ids = [
    "bcca318e644999583156ff3ab9de3d6bd02295dcb166e39006568ffe9582207b",
    "edb64996c64421378537d941a77426a5a7bd84f9967ad9f523ca61386e5469ed",
    "5f96908e3ce5d84d693ddd79670d60cd1c6a02646aa779f46586a7699dda7c25",
    "0f137538a170b64258c0e10839cad325e5f22fc5bb5fa9e20b7351f0a680366",
]
assert (
```

```
df.filter(pl.col("id").is_in(ids)).sort("id")[col_name] == [2, 1, 1, 1]
).all()
```

2. [2 балла] Распределите студентов в соответствии с первым приоритетом

```
In [30]: special_students = df.filter(pl.col('fall_1') == pl.col('fall_2'), pl.col('cours
special_students
```

Out[30]: shape: (1, 16)

	id	timestamp	rating	group_22	fall_1
	str	str	i64	i64	str
	"532a78c1cc336e7ecb1d4189f1e249..."	"2024-08-14 17:01:04"	1270	0	"Байесовские методы в машинном ..."



```
In [31]: df_temp = df.with_columns(
    pl.when(pl.col('id').is_in(special_students['id']))
    .then(pl.lit(''))
    .otherwise(pl.col('fall_2'))
    .alias('fall_2')
)
```

C:\Users\8dant\AppData\Local\Temp\ipykernel_7628\2390732266.py:1: DeprecationWarning: `is\_in` with a collection of the same datatype is ambiguous and deprecated. Please use `implode` to return to previous behavior.

See <https://github.com/pola-rs/polars/issues/22149> for more information.

```
df_temp = df.with_columns(
```

```
In [32]: gigachads = df_temp.filter(
    (pl.col('fall_1') == '') | (pl.col('fall_2') == '')
)

gigachads
```

Out[32]: shape: (1, 16)

	id	timestamp	rating	group_22	fall_1	fall_2
	str	str	i64	i64	str	str
	"532a78c1cc336e7ecb1d4189f1e249..."	"2024-08-14 17:01:04"	1270	0	"Байесовские методы в машинном ..."	""



```
In [33]: df_mops = df_temp.filter(
    pl.col('courses_count') == 2
)

df_bros = df_temp.filter(
```

```
pl.col('courses_count') == 1
)
```

```
In [34]: def wave_simulator(da, col1, col2=''):
    if col2 != '':
        df_students = da.select(pl.col('id'), pl.col(col1), pl.col(col2), pl.col('percentile'))
        df1 = (
            df_students
            .melt(id_vars=['id', 'percentile'], value_vars=[col1, col2])
            .rename({'value': 'course'})
            .drop('variable')
        )
    else:
        df1 = da.select(pl.col('id'), pl.col(col1), pl.col('percentile')).rename({'value': 'course'})

    df_filtered = df1.filter(pl.col('course') != '')
    df_sorted = df_filtered.sort(['course', 'percentile'], descending=[False, False])

    result = (
        df_sorted
        .group_by('course')
        .agg(
            pl.count().alias('applicants'),
            pl.col('id').alias('ids'),
            pl.col('percentile').alias('percentiles')
        )
    )

    return result
```

```
In [35]: res_mops = wave_simulator(df_mops, 'fall_1', 'fall_2')
        res_bros = wave_simulator(df_bros, 'fall_1')
```

C:\Users\8dant\AppData\Local\Temp\ipykernel_7628\2678280930.py:6: DeprecationWarning: `DataFrame.melt` is deprecated; use `DataFrame.unpivot` instead, with `index` instead of `id\_vars` and `on` instead of `value\_vars`
 .melt(id_vars=['id', 'percentile'], value_vars=[col1, col2])
C:\Users\8dant\AppData\Local\Temp\ipykernel_7628\2678280930.py:20: DeprecationWarning: `pl.count()` is deprecated. Please use `pl.len()` instead.
(Deprecated in version 0.20.5)
 pl.count().alias('applicants'),

```
In [36]: def collect(df_dogs, df_people):
    data = pl.concat([
        df_dogs.select('course', 'ids', 'percentiles'),
        df_people.select('course', 'ids', 'percentiles')
    ], how='vertical')

    data = data.explode(['ids', 'percentiles']).sort(['course', 'percentiles'], descending=[False, False])

    result = (
        data
        .group_by('course')
        .agg(
            pl.count().alias('total'),
            pl.col('ids').alias('ids'),
            pl.col('percentiles').alias('percentiles')
        )
    )
```

```
)  
  
return result
```

```
In [37]: res = collect(res_mops, res_bros)  
  
result = res.with_columns(  
    pl.when(  
        pl.col('course').is_in([  
            'Количественные финансы',  
            'Промышленное программирование на Haskell',  
            'Рекомендательные системы'  
        ])  
    )  
    .then(60)  
    .when(pl.col('course') == 'Глубинное обучение в обработке звука')  
    .then(1000)  
    .otherwise(30)  
    .alias('maximum')  
)  
  
result
```

```
C:\Users\8dant\AppData\Local\Temp\ipykernel_7628\598973891.py:13: DeprecationWarning: `pl.count()` is deprecated. Please use `pl.len()` instead.  
(Deprecated in version 0.20.5)  
    pl.count().alias('total'),
```

Out[37]: shape: (26, 5)

course	total	
		str u32
"Self-supervised Learning"	18	["92f4503e31c17e957f5c5ce045c496f440b0565d37f9aa6118f3e52e "f583485bad0154b01330fee4f80b91e28147483f3ff8719c20d8ec856e "ebc2505ac729ff1cc6c5fb016d061ad6f9f0ffff16b7fc7faf5d2b3a
"Statistical Learning Theory"	26	["4bf283dde0d10c6e0d4dd9f414dbccc2013136691632d0e3a7ec92af "c34b8eccee3b21a7270de57809f80add8ed01a45be756508ff362fc4b0 "6fa015d8846e21f4f79bac013b469a1e6e962fffd6121b4d9bcde390
"Байесовские методы в машинном ..."	31	["2887dd24eabdc36c2c0c2f663ff6b63f4ce36ca9aea880483dbf123 "cafbec4fff4c9e69ac9a5497c07734373444d377c9eb424bcf109f6b83 "6fa015d8846e21f4f79bac013b469a1e6e962fffd6121b4d9bcde390
"Безопасность компьютерных сист..."	18	["6e1190bb70002259960601e0c4984c1c068a2d88c508f546a5f9dea6 "6828d0f6015f0cdead83f4aa7277908a5ec829e9d4c351e219421adb "b9f5a2998bac784b392c7d70268759ca4c1a1838a0af82b65b53f727
"Введение в платформы данных"	21	["69c23d81afb1431da28fdec9e1edf407dbca5ab0fab273826634114d "60553144cbe31126977f8493bb151028cc76e0112f543730075380d "f000c323c181779ceea1e21b7063e744a5aa023441efe6be97c61abe
...	...	
"Трансформерные и мультимодальн..."	9	["1a839aea491d75693c030b712452dcd9215457a97d2f86c6c9cdfcf7 "4bf710d7b89064e1a3cbcc69b7ca67c20d6181dc72ad3592039d366d "79c832498df0572a38f095bd7d11d1b9cb724da6c2ffb693d6ed2aa6
"Функциональное программировани..."	4	["6f55e5871cd9cffb3fba8b1606eb5e33b42ebacc17b8e19d62514e31 "5e146e05e41c9839e80b70d892167312585069e635f333cf59263ee2 "9ce7660425b7435d3f5f215c3cd44e5dd1d3bcd87e46ffefd3e8d460
"Эксплуатация и надёжность прог..."	1	["62c5f09073a6e6cdf1dbcf39c55b78a69d12f31798db5fcd3a824f4a
"Язык SQL"	37	["79da3965a7836de9a314e56f4f632e7103ce3af0af5389f11bf6086d "76173ca6253bbea653ac5a08a6deec665b56277a20deae5a6eb53a32 "9804b2d1ad973243d05cab548021c13fc50c73cc41f56edd4090438
"Язык программирования Scala"	1	["ad4f2591f6c525b01b4267b293784e63b75e769020769c5b8b7b0d29



```
In [38]: def distribute(dfr, selection):  
         students = dfr.select('id', 'courses_count')  
  
         course_students = (  
             selection  
             .explode(['ids', 'percentiles'])
```

```

        .with_columns(
            pl.col('ids').alias('id'),
            pl.col('percentiles').alias('percentile')
        )
        .select('course', 'maximum', 'id', 'percentile')
        .sort(['course', 'percentile'], descending=[False, False])
    )

    ranked = (
        course_students
        .with_columns(
            pl.col('percentile')
            .rank(method='ordinal', descending=False)
            .over('course')
            .alias('rank')
        )
        .filter(pl.col('rank') <= pl.col('maximum'))
        .select('course', 'id')
    )

    student_courses = (
        ranked
        .group_by('id')
        .agg(
            pl.col('course').alias('courses'),
        )
    )

    result = (
        students
        .join(student_courses, on='id', how='left')
        .with_columns(
            pl.when(pl.col('courses').list.len() > 0)
            .then(pl.col('courses').list.first())
            .otherwise(None).alias('course1'),
            pl.when(pl.col('courses').list.len() > 1)
            .then(pl.col('courses').list.last())
            .otherwise(None).alias('course2')
        )
        .select('id', 'course1', 'course2', 'courses_count')
    )

    return result

```

```

In [39]: df_wave_1 = distribute(df_temp, result)
df_wave_1 = df_wave_1.rename({'id' : 'ID', 'courses_count' : 'fall_course_num'})

```

```

In [40]: df_pandas = df_wave_1.to_pandas()

```

Сверьте назначенные вами на первой волне курсы с эталоном. Для этого преобразуйте данные о назначенных курсах, приведя их в разрез "по студенту" (подробнее см в докстринге тест-функции), **переведите в pandas.DataFrame** и передайте в функцию 'check_first_wave_assignments'. Если функция не выбросила Exception, то всё хорошо!

Обратите внимание: здесь обрабатывать детали условия касающиеся ответов '???' и '-' из требований к решению обрабатывать не нужно!

Трансформацию датасета (создание колонок 'course1' и 'course2') советуем выделить в отдельную функцию, поскольку она понадобится вам далее при сдаче результатов в тестирующую систему.

```
In [41]: import pandas as pd
# !wget -O 'task_2_check.csv' -q 'https://www.dropbox.com/scl/fi/mxmp3b8ecssaei
check_df = pd.read_csv('./task_2_check.csv')
```

Если открытие check_df кинуло Exception, проверьте, что файл лежит в той же директории, из которой запущено ядро (kernel) вашего ноутбука (в его current working directory).

Стандартно файл должен скачиваться как раз туда, но вдруг у вас что-то пошло не так

```
In [42]: from pandas.testing import assert_frame_equal
from collections import Counter

def check_first_wave_assignments(assignments: pd.DataFrame) -> None:
    """
    assignments - pandas DataFrame с колонками ['ID', 'course1', 'course2', 'fall_course_num']
    - 'ID' - id студента (не объект индекса!)
    - 'course1', 'course2' - названия курсов, назначенные студенту в первую волну
    Если один из курсов не назначен (вне зависимости от того, должен ли был быть
    - 'fall_course_num' - число осенних курсов, которые должен выбрать студент.
    """

    def course_columns_to_set_column(row: pd.DataFrame):
        result = Counter()

        for column in ['course1', 'course2']:
            if (not pd.isna(row[column]) and row[column] is not None):
                result[row[column]] += 1
        return result

    ANSWER_SOURCE = 'task_2_check.csv'

    ALTERNATIVES = {
        "cdb87f7b15495e514eb481230b44540c6b0cfeb347c991f06b1e361fcdef0c0b": [
            Counter({"Глубинное обучение для текстовых данных", "Байесовские методы в машинном обучении"}),
            Counter({"Глубинное обучение для текстовых данных", "Байесовские методы в машинном обучении"}),
        ],
        "6fa015d8846e21f4f79bac013b469a1e6e962fffd6121b4d9bcde3906275293e": [
            Counter({"Statistical Learning Theory", "Байесовские методы в машинном обучении"}),
            Counter({"Statistical Learning Theory", "Байесовские методы в машинном обучении"}),
        ]
    }

    answer_df = pd.read_csv(ANSWER_SOURCE)
    assignments = assignments.copy()
    assignments['course_set'] = assignments.apply(course_columns_to_set_column, axis=1)
    answer_df['course_set'] = answer_df.apply(course_columns_to_set_column, axis=1)

    answer_df_common = answer_df[~answer_df['ID'].isin(ALTERNATIVES.keys())][['ID', 'course_set']]
    assignments_common = assignments[~assignments['ID'].isin(ALTERNATIVES.keys())][['ID', 'course_set']]

    assert_frame_equal(answer_df_common, assignments_common)
```

```

answer_df_common = answer_df_common.set_index('ID').sort_index()
assignments_common = assignments_common.set_index('ID').sort_index()

assert_frame_equal(answer_df_common, assignments_common, check_dtype=False,

for student in ALTERNATIVES:
    assert assignments[assignments['ID'] == student]['course_set'].isin(ALTE
    assert answer_df[answer_df['ID'] == student]['fall_course_num'].iloc[0]

```

In [43]: `check_first_wave_assignments(df_pandas)`

Тут может выдаваться ошибка, но проблему я описал ранее, если код корректно запускать (в нужном порядке, где точно не делаются никакие запросы и оптимизации), то все работает

3. [2 балла] Проведите все три волны отбора студентов на курсы по выбору

In [44]: `df_part = df_wave_1`

In [45]: `df_wave_1.filter(pl.col('fall_course_num') == 2).filter(pl.col('course1').is_nul`

Out[45]: shape: (4, 4)

ID	course1	course2	fall_course_num
str	str	str	i32
"f9ea5e817d10c1450d85bc1429dfc5...	"Введение в платформы данных"	null	2
"776f50522ff3139728cb2ee5721708...	"Байесовские методы в машинном ..."	null	2
"6fa015d8846e21f4f79bac013b469a...	"Statistical Learning Theory"	null	2
"532a78c1cc336e7ecb1d4189f1e249...	"Байесовские методы в машинном ..."	null	2

Тогда для второй волны мы всегда будем смотреть только на одну колонку:

1. Если чел с `fall_course_num = 1`, то `fall_2`
2. Если чел с `fall_course_num = 2`, то `fall_3`

```

In [46]: rest1 = df_wave_1.filter(pl.col('course1').is_null()).filter(pl.col('fall_course
rest2 = df_wave_1.filter(pl.col('course2').is_null()).filter(pl.col('fall_course

rest1 = rest1.rename({'ID' : 'id'})
rest2 = rest2.rename({'ID' : 'id'})

rest = (
    pl.concat([
        rest1,
        rest2
    ])
)

```



```

check = df_temp.join(
    rest,
    on='id',
    how='inner'
)

rest = check
rest.filter(pl.col('id').is_in(rest2['id']))

```

C:\Users\8dant\AppData\Local\Temp\ipykernel_7628\665694967.py:21: DeprecationWarning: `is\_in` with a collection of the same datatype is ambiguous and deprecated. Please use `implode` to return to previous behavior.

See <https://github.com/pola-rs/polars/issues/22149> for more information.
rest.filter(pl.col('id').is_in(rest2['id']))

Out[46]: shape: (4, 19)

	id	timestamp	rating	group_22	fall_1	
	str	str	i64	i64	str	
"f9ea5e817d10c1450d85bc1429dfc5..."		"2024-08-13 16:16:55"	1104	0	"Дизайн систем"	"Введе платф да
"776f50522ff3139728cb2ee5721708..."		"2024-08-14 18:40:34"	1022	0	"Дизайн систем"	"Байесо мет машин
"6fa015d8846e21f4f79bac013b469a..."		"2024-08-15 10:54:22"	897	0	"Statistical Learning Theory"	"Байесо мет машин
"532a78c1cc336e7ecb1d4189f1e249..."		"2024-08-14 17:01:04"	1270	0	"Байесовские методы в машинном ..."	

Раз тут нет таких остатков, что fall_3 это что-то, что уже выбирали в первой волне, то не буду доп фильтр писать

```

In [47]: def calculate_max(dataframe):
    temp = pl.concat([
        dataframe.select(course=pl.col('course1')),
        dataframe.select(course=pl.col('course2'))
    ]).drop_nulls()

    result = (
        temp.group_by('course')
        .agg(taken=pl.count())
        .with_columns(
            maximum=pl.when(
                pl.col('course').is_in([
                    'Количественные финансы',
                    'Промышленное программирование на Haskell',
                    'Рекомендательные системы'
                ])
            )
    )

```

```

        .then(60)
        .when(pl.col('course') == 'Глубинное обучение в обработке звука')
        .then(1000)
        .otherwise(30)
    )
    .with_columns(
        remaining=pl.col('maximum') - pl.col('taken')
    )
)
return result

```

```

In [48]: rest1 = rest.join(
    rest1,
    on='id',
    how='inner'
)
rest2 = rest.join(
    rest2,
    on='id',
    how='inner'
)

```

```

In [49]: res1 = wave_simulator(rest1, 'fall_2')
res2 = wave_simulator(rest2, 'fall_3')

res = collect(res1, res2)

```

C:\Users\8dant\AppData\Local\Temp\ipykernel_7628\2678280930.py:20: DeprecationWarning: `pl.count()` is deprecated. Please use `pl.len()` instead.
(Deprecated in version 0.20.5)
pl.count().alias('applicants'),
C:\Users\8dant\AppData\Local\Temp\ipykernel_7628\598973891.py:13: DeprecationWarning: `pl.count()` is deprecated. Please use `pl.len()` instead.
(Deprecated in version 0.20.5)
pl.count().alias('total'),

```

In [50]: new_max = calculate_max(df_wave_1)
res = res.join(
    new_max.select('course', 'remaining'),
    on='course',
    how='left'
)

res = res.with_columns(
    pl.when(pl.col('remaining').is_null())
    .then(30)
    .otherwise(pl.col('remaining'))
    .alias('maximum')
)

```

C:\Users\8dant\AppData\Local\Temp\ipykernel_7628\575110916.py:9: DeprecationWarning: `pl.count()` is deprecated. Please use `pl.len()` instead.
(Deprecated in version 0.20.5)
.agg(taken=pl.count())

```

In [51]: df_wave_2 = distribute(rest, res)
df_wave_2 = df_wave_2.rename({'id' : 'ID', 'courses_count' : 'fall_course_num'})

```

```

In [52]: df_part = df_part.join(
          df_wave_2.select(['ID', 'course1']).rename({'course1': 'new_course'}),
          on='ID',
          how='left'
        )

df_part = df_part.with_columns(
    pl.when(
        (pl.col('new_course').is_not_null()) &
        (pl.col('course1').is_null())
    )
    .then(pl.col('new_course'))
    .otherwise(pl.col('course1'))
    .alias('course1'),
)

df_part = df_part.with_columns(
    pl.when(
        (pl.col('new_course').is_not_null()) &
        (pl.col('new_course') != pl.col('course1')) &
        (pl.col('course2').is_null())
    )
    .then(pl.col('new_course'))
    .otherwise(pl.col('course2'))
    .alias('course2')
)

df_part = df_part.drop('new_course')

```

```
In [53]: df_part_2 = df_part
```

```
In [54]: df_part.filter(pl.col('fall_course_num') == 2).filter(pl.col('course1').is_null())
```

```
Out[54]: shape: (2, 4)
```

	ID	course1	course2	fall_course_num
	str	str	str	i32
	"776f50522ff3139728cb2ee5721708..."	"Байесовские методы в машинном ..."	null	2
	"532a78c1cc336e7ecb1d4189f1e249..."	"Байесовские методы в машинном ..."	null	2

```

In [55]: final = df_part.filter(pl.col('course1').is_null()).filter(pl.col('fall_course_n

final = final.rename({'ID' : 'id'})

check = df_temp.join(
    final,
    on='id',
    how='inner'
)

final = check

```

```
In [56]: fin = wave_simulator(final, 'fall_3')
```

C:\Users\8dant\AppData\Local\Temp\ipykernel_7628\2678280930.py:20: DeprecationWarning: `pl.count()` is deprecated. Please use `pl.len()` instead.
(Deprecated in version 0.20.5)
pl.count().alias('applicants'),

```
In [57]: fin_max = calculate_max(df_part)
fin = fin.join(
    fin_max.select('course', 'remaining'),
    on='course',
    how='left'
)

fin = fin.with_columns(
    pl.when(pl.col('remaining').is_null())
    .then(30)
    .otherwise(pl.col('remaining'))
    .alias('maximum')
)
```

C:\Users\8dant\AppData\Local\Temp\ipykernel_7628\575110916.py:9: DeprecationWarning: `pl.count()` is deprecated. Please use `pl.len()` instead.
(Deprecated in version 0.20.5)
.agg(taken=pl.count())

```
In [58]: df_wave_3 = distribute(final, fin)
df_wave_3 = df_wave_3.rename({'id' : 'ID', 'courses_count' : 'fall_course_num'})
```

```
In [59]: df_part = df_part.join(
    df_wave_3.select(['ID', 'course1']).rename({'course1': 'new_course'}),
    on='ID',
    how='left'
)

df_part = df_part.with_columns(
    pl.when(
        (pl.col('new_course').is_not_null()) &
        (pl.col('course1').is_null())
    )
    .then(pl.col('new_course'))
    .otherwise(pl.col('course1'))
    .alias('course1'),
)

df_part = df_part.with_columns(
    pl.when(
        (pl.col('new_course').is_not_null()) &
        (pl.col('new_course') != pl.col('course1')) &
        (pl.col('course2').is_null())
    )
    .then(pl.col('new_course'))
    .otherwise(pl.col('course2'))
    .alias('course2')
)

df_part = df_part.select('ID', 'course1', 'course2', 'fall_course_num')
```

```
In [60]: df_part
```

Out[60]: shape: (418, 4)

	ID	course1	course2	fall_course_num
	str	str	str	i32
"b8ab1a4ad4926caca7b82f8d1d11b2..."	"Дизайн систем"		null	1
"40c3044ac2c56f3b576601f7052685..."	"Разработка микросервисов на Go"		null	1
"15a368d1368e4fd9b6dc0f2bc9362f..."	"Основы разработки компьютерных..."		null	1
"e6b3b61c7e6a0dcdd69e96325d12b2..."	"Основы разработки компьютерных..."		null	1
"274f10a994f43939375bdfb3b3ac23..."	"Прикладная статистика в машинн..."		null	1
...	...	...	...	...
"3501992c6601498ed885bfbb2deefe..."	"Байесовские методы в машинном ..."	"Генеративные модели на основе ..."		2
"48fa126659b29ec4bfcd13cbc83506..."	"Дизайн систем"		null	1
"edb64996c64421378537d941a77426..."	"Операционные системы 2"		null	1
"d41f00051219658f10ba5bd41aba05..."	"Распределенные системы"		null	1
"8edbf92455432dd49c7966b801be12..."	"Распределенные системы"		null	1

```
In [61]: df_part = df_part.with_columns(  
    pl.when(pl.col('fall_course_num') == 1)  
    .then(pl.lit('-'))  
    .otherwise('course2')  
    .alias('course2')  
)
```

```
In [62]: df_part = df_part.with_columns(  
    pl.when(pl.col('fall_course_num') == 1, pl.col('course1').is_null())  
    .then(pl.lit('???'))  
    .otherwise('course1')  
    .alias('course1'),  
  
    pl.when(pl.col('fall_course_num') == 2, pl.col('course2').is_null())  
    .then(pl.lit('???'))  
    .otherwise('course2')  
    .alias('course2')  
)
```

```
In [63]: df_part = df_part.drop('fall_course_num')
```

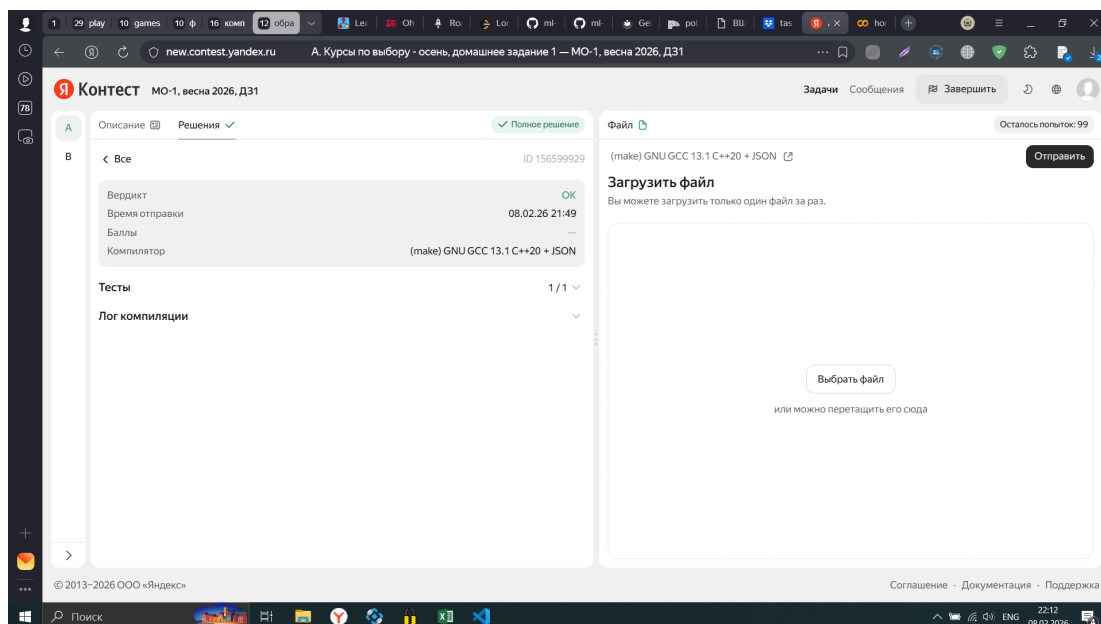
```
In [64]: df_part
```

```
Out[64]: shape: (418, 3)
```

ID	course1	course2
str	str	str
"b8ab1a4ad4926caca7b82f8d1d11b2..."	"Дизайн систем"	"_"
"40c3044ac2c56f3b576601f7052685..."	"Разработка микросервисов на Go"	"_"
"15a368d1368e4fd9b6dc0f2bc9362f..."	"Основы разработки компьютерных..."	"_"
"e6b3b61c7e6a0dcdd69e96325d12b2..."	"Основы разработки компьютерных..."	"_"
"274f10a994f43939375bdfb3b3ac23..."	"Прикладная статистика в машинн..."	"_"
...	...	...
"3501992c6601498ed885bfbb2deefe..."	"Байесовские методы в машинном ..."	"Генеративные модели на основе ..."
"48fa126659b29ec4bfcd13cbc83506..."	"Дизайн систем"	"_"
"edb64996c64421378537d941a77426..."	"Операционные системы 2"	"_"
"d41f00051219658f10ba5bd41aba05..."	"Распределенные системы"	"_"
"8edbf92455432dd49c7966b801be12..."	"Распределенные системы"	"_"

```
In [65]: df_part.write_csv('solution.csv')
```

Отправьте свой файл `res_fall.csv` в [Я.контекст](#) и прикрепите/укажите ниже ваш никнейм и id успешной посылки.



Даниил Перевалов dvperevalov@edu.hse.ru ID 156599929

Дисклеймер:

Контеcт выдаётся для самопроверки. Если ваша посылка получила ОК, то код, скорее всего, правильный. Но при этом оценка всё равно может быть снижена в случае обнаружения неэффективностей или ошибок в коде. Если вы сдадите в AnyTask очевидно неработающий код или ноутбук без кода, но при этом в контеcт будет сдан корректный файл, то это будет расцениваться как плагиат.

На всякий случай просим вас сдать вместе с ноутбуком файл `res_fall.csv` в anytask

4. [1 балл] Распределите таким же образом студентов еще и на весенние курсы по выбору.

Если ваш код был хорошо структурирован, то это не составит проблем.

Если вы выполнили это задание, сдайте среди прочего файл `res_spring.csv` в таком же формате, как и `res_fall.csv`.

```
In [66]: df_temp = df_temp.select('id', 'spring_1', 'spring_2', 'spring_3', 'percentile',
df_temp = df_temp.with_columns(
    pl.when(pl.col('courses_count') == 2)
      .then(True)
      .otherwise('is_ml_student')
      .alias('is_ml_student')
)
```

```
In [67]: df_temp.filter(pl.col('spring_1') == 'Машинное обучение 2').filter(pl.col('is_ml
```

Out[67]: shape: (0, 9)

id	spring_1	spring_2	spring_3	percentile	is_ml_student	courses_count	group_22	g
str	str	str	str	f64	bool	i32	i64	



```
In [68]: df_temp.filter(pl.col('spring_2') == 'Машинное обучение 2').filter(pl.col('is_ml
```

Out[68]: shape: (0, 9)

id	spring_1	spring_2	spring_3	percentile	is_ml_student	courses_count	group_22	g
str	str	str	str	f64	bool	i32	i64	



```
In [69]: special = df_temp.filter(pl.col('spring_3') == 'Машинное обучение 2').filter(pl.  
special
```

Out[69]: shape: (2, 9)

id	spring_1	spring_2	spring_3	perc
str	str	str	str	
"7935540b095acdf82e4baf21202af7..."	"Стохастический анализ"	"Генеративные модели в машинном..."	"Машинное обучение 2"	0.9
"6a43e43c97292ddc20ee6204067168..."	"Генеративные модели в машинном..."	"Компьютерные сети"	"Машинное обучение 2"	0.9



Для вот этих двоих обнулим последний приоритет, поскольку нельзя подаваться на машинку 2 мопам.

```
In [70]: df_temp = df_temp.with_columns(  
    pl.when(pl.col('id').is_in(special['id']))  
    .then(pl.lit(''))  
    .otherwise(pl.col('spring_3'))  
    .alias('spring_3')  
)
```

C:\Users\8dant\AppData\Local\Temp\ipykernel_7628\2216446998.py:1: DeprecationWarning: `is\_in` with a collection of the same datatype is ambiguous and deprecated. Please use `implode` to return to previous behavior.

See <https://github.com/pola-rs/polars/issues/22149> for more information.
df_temp = df_temp.with_columns(
 pl.when(pl.col('id').is_in(special['id']))
 .then(pl.lit(''))
 .otherwise(pl.col('spring_3'))
 .alias('spring_3')
)

```
In [71]: special_students = df_temp.filter(pl.col('spring_1') == pl.col('spring_2'), pl.c  
df_temp = df_temp.with_columns(  
    pl.when(pl.col('id').is_in(special_students['id']))  
    .then(pl.lit(''))  
    .otherwise(pl.col('spring_2'))  
    .alias('spring_2')  
)
```


C:\Users\8dant\AppData\Local\Temp\ipykernel_7628\1861060724.py:2: DeprecationWarning: `is\_in` with a collection of the same datatype is ambiguous and deprecated. Please use `implode` to return to previous behavior.

See <https://github.com/pola-rs/polars/issues/22149> for more information.

```
df_temp = df_temp.with_columns(
```

```
In [72]: gigachads = df_temp.filter(
          (pl.col('spring_1') == '') & (pl.col('spring_2') == ''))
          )
          gigachads
```

Out[72]: shape: (45, 9)

	id	spring_1	spring_2	spring_3	percentile	is_ml_stud
	str	str	str	str	f64	bool
	"330df17dbb7c4f0651d6ced20f90d8...	""	""	""	0.025926	fa
	"ba6aeec35c518ef0ef22a92dff1e65...	""	""	""	0.77963	fa
	"12cf8adb8993c25f276cfa02995581...	""	""	""	0.431481	fa
	"48063f2b2d60f5a28901923e86746f...	""	""	""	0.161111	fa
	"79da3965a7836de9a314e56f4f632e...	""	""	""	0.011111	fa
	...	...	...	...	...	
	"e4af61a6a57e0544d5b9bbf9572cd7...	""	""	""	0.496296	fa
	"1ac8c3290d194ea958a120a9546b44...	""	""	""	0.411111	fa
	"ddaf2a5025a0b6df600db0a20fc79f...	""	""	""	0.598148	fa
	"a887673b45ddf0c8473210e11a0b86...	""	""	""	0.424074	fa
	"8edbf92455432dd49c7966b801be12...	""	""	""	0.190741	fa

```
In [73]: df_mops = df_temp.filter(
          pl.col('courses_count') == 2
          )

          df_bros = df_temp.filter(
          pl.col('courses_count') == 1
          )

          res_mops = wave_simulator(df_mops, 'spring_1', 'spring_2')

          res_bros = wave_simulator(df_bros, 'spring_1')

          res = collect(res_mops, res_bros)

          result = res.with_columns(
            pl.when(
              pl.col('course').is_in([
                'Количественные финансы',
                'Промышленное программирование на Haskell',
                'Рекомендательные системы'
```

```

    ])
)
    .then(60)
    .when(pl.col('course') == 'Глубинное обучение в обработке звука')
    .then(1000)
    .otherwise(30)
    .alias('maximum')
)

df_wave_1 = distribute(df_temp, result)
df_wave_1 = df_wave_1.rename({'id' : 'ID', 'courses_count' : 'fall_course_num'})

df_part = df_wave_1

```

C:\Users\8dant\AppData\Local\Temp\ipykernel_7628\2678280930.py:6: DeprecationWarning: `DataFrame.melt` is deprecated; use `DataFrame.unpivot` instead, with `index` instead of `id\_vars` and `on` instead of `value\_vars`
 .melt(id_vars=['id', 'percentile'], value_vars=[col1, col2])
C:\Users\8dant\AppData\Local\Temp\ipykernel_7628\2678280930.py:20: DeprecationWarning: `pl.count()` is deprecated. Please use `pl.len()` instead.
(Deprecated in version 0.20.5)
 pl.count().alias('applicants'),
C:\Users\8dant\AppData\Local\Temp\ipykernel_7628\598973891.py:13: DeprecationWarning: `pl.count()` is deprecated. Please use `pl.len()` instead.
(Deprecated in version 0.20.5)
 pl.count().alias('total'),

In [74]: `df_wave_1.filter(pl.col('fall_course_num') == 2).filter(pl.col('course1').is_nul`

Out[74]: shape: (21, 4)

ID	course1	course2	fall_course_num
str	str	str	i32
"52a0895a22e19ab73c67a97f91c4e2..."	"Методы сжатия и передачи медиа..."	null	2
"f9ea5e817d10c1450d85bc1429dfc5..."	"Эффективные системы глубинного..."	null	2
"776f50522ff3139728cb2ee5721708..."	"Эффективные системы глубинного..."	null	2
"0d223588b0f2bc50094a0a577299a1..."	null	null	2
"ebc2505ac729ff1cc6c5fb016d061a..."	"Стохастический анализ"	null	2
...	...	...	...
"acb47d76f3aa4524fba24b7e9feabe..."	"Современные языковые модели"	null	2
"c7fd0f0963f73bd01f3540d20e24f0..."	"Современные языковые модели"	null	2
"1107cb34832723b91e2dee89481a99..."	"Развёртывание ML-моделей в выс..."	null	2
"f3850b3aa2c8c622ee28bb6368a9b8..."	"Теория и практика онлайн-экспе..."	null	2
"b315b92935edc5a3e9286170b5a534..."	"Математические основы нейросет..."	null	2

```
In [75]: rest1 = df_wave_1.filter(pl.col('course1').is_null()).filter(pl.col('fall_course')
rest2 = df_wave_1.filter(pl.col('course2').is_null()).filter(pl.col('fall_course')

rest1 = rest1.rename({'ID' : 'id'})
rest2 = rest2.rename({'ID' : 'id'})

rest = (
    pl.concat([
        rest1,
        rest2
    ])
)

check = df_temp.join(
    rest,
    on='id',
    how='inner'
)

rest = check
rest1 = rest.join(
    rest1,
    on='id',
    how='inner'
)
```

```

rest2 = rest.join(
    rest2,
    on='id',
    how='inner'
)

res1 = wave_simulator(rest1, 'spring_2')
res2 = wave_simulator(rest2, 'spring_3')

res = collect(res1, res2)

new_max = calculate_max(df_wave_1)
res = res.join(
    new_max.select('course', 'remaining'),
    on='course',
    how='left'
)

res = res.with_columns(
    pl.when(pl.col('remaining').is_null())
    .then(30)
    .otherwise(pl.col('remaining'))
    .alias('maximum')
)

df_wave_2 = distribute(rest, res)
df_wave_2 = df_wave_2.rename({'id' : 'ID', 'courses_count' : 'fall_course_num'})

```

C:\Users\8dant\AppData\Local\Temp\ipykernel_7628\2678280930.py:20: DeprecationWarning: `pl.count()` is deprecated. Please use `pl.len()` instead.
(Deprecated in version 0.20.5)
pl.count().alias('applicants'),
C:\Users\8dant\AppData\Local\Temp\ipykernel_7628\598973891.py:13: DeprecationWarning: `pl.count()` is deprecated. Please use `pl.len()` instead.
(Deprecated in version 0.20.5)
pl.count().alias('total'),
C:\Users\8dant\AppData\Local\Temp\ipykernel_7628\575110916.py:9: DeprecationWarning: `pl.count()` is deprecated. Please use `pl.len()` instead.
(Deprecated in version 0.20.5)
.agg(taken=pl.count())

```

In [76]: df_part = df_part.join(
    df_wave_2.select(['ID', 'course1']).rename({'course1': 'new_course'}),
    on='ID',
    how='left'
)

df_part = df_part.with_columns(
    pl.when(
        (pl.col('new_course').is_not_null()) &
        (pl.col('course1').is_null())
    )
    .then(pl.col('new_course'))
    .otherwise(pl.col('course1'))
    .alias('course1'),
)

df_part = df_part.with_columns(
    pl.when(

```

```

        (pl.col('new_course').is_not_null()) &
        (pl.col('new_course') != pl.col('course1')) &
        (pl.col('course2').is_null())
    )
    .then(pl.col('new_course'))
    .otherwise(pl.col('course2'))
    .alias('course2')
)

df_part = df_part.drop('new_course')

```

```

In [77]: df_part_2 = df_part
df_part.filter(pl.col('fall_course_num') == 2).filter(pl.col('course1').is_null(

```

Out[77]: shape: (5, 4)

	ID	course1	course2	fall_course_num
	str	str	str	i32
"0d223588b0f2bc50094a0a577299a1..."	"Эффективные системы глубинного..."		null	2
"ebc2505ac729ff1cc6c5fb016d061a..."	"Стохастический анализ"		null	2
"4aec9a30868dc637f1e360717edf3c..."	"Методы сжатия и передачи медиа..."		null	2
"6fa015d8846e21f4f79bac013b469a..."	"Математические основы нейросет..."		null	2
"c7fd0f0963f73bd01f3540d20e24f0..."	"Современные языковые модели"		null	2

```

In [78]: final = df_part.filter(pl.col('course1').is_null()).filter(pl.col('fall_course_n

final = final.rename({'ID' : 'id'})

check = df_temp.join(
    final,
    on='id',
    how='inner'
)

final = check

fin = wave_simulator(final, 'spring_3')

fin_max = calculate_max(df_part)
fin = fin.join(
    fin_max.select('course', 'remaining'),
    on='course',
    how='left'
)

fin = fin.with_columns(
    pl.when(pl.col('remaining').is_null())
    .then(30)

```

```

        .otherwise(pl.col('remaining'))
        .alias('maximum')
    )

df_wave_3 = distribute(final, fin)
df_wave_3 = df_wave_3.rename({'id' : 'ID', 'courses_count' : 'fall_course_num'})

```

C:\Users\8dant\AppData\Local\Temp\ipykernel_7628\2678280930.py:20: DeprecationWarning: `pl.count()` is deprecated. Please use `pl.len()` instead.
(Deprecated in version 0.20.5)
 pl.count().alias('applicants'),
C:\Users\8dant\AppData\Local\Temp\ipykernel_7628\575110916.py:9: DeprecationWarning: `pl.count()` is deprecated. Please use `pl.len()` instead.
(Deprecated in version 0.20.5)
 .agg(taken=pl.count())

```

In [79]: df_part = df_part.join(
        df_wave_3.select(['ID', 'course1']).rename({'course1': 'new_course'}),
        on='ID',
        how='left'
    )

df_part = df_part.with_columns(
    pl.when(
        (pl.col('new_course').is_not_null()) &
        (pl.col('course1').is_null())
    )
    .then(pl.col('new_course'))
    .otherwise(pl.col('course1'))
    .alias('course1'),
)

df_part = df_part.with_columns(
    pl.when(
        (pl.col('new_course').is_not_null()) &
        (pl.col('new_course') != pl.col('course1')) &
        (pl.col('course2').is_null())
    )
    .then(pl.col('new_course'))
    .otherwise(pl.col('course2'))
    .alias('course2')
)

df_part = df_part.select('ID', 'course1', 'course2', 'fall_course_num')

```

```

In [80]: df_part = df_part.with_columns(
        pl.when(pl.col('fall_course_num') == 1)
        .then(pl.lit('-'))
        .when((pl.col('fall_course_num') == 2) & (pl.col('course2').is_null()))
        .then(pl.lit('???'))
        .otherwise(pl.col('course2'))
        .alias('course2')
    )

df_part = df_part.with_columns(
    pl.when(((pl.col('fall_course_num') == 1) | (pl.col('fall_course_num') == 2)
    .then(pl.lit('???'))
    .otherwise(pl.col('course1'))
    .alias('course1')
)
)

```

```
df_part = df_part.drop('fall_course_num')
```

```
In [81]: df_part
```

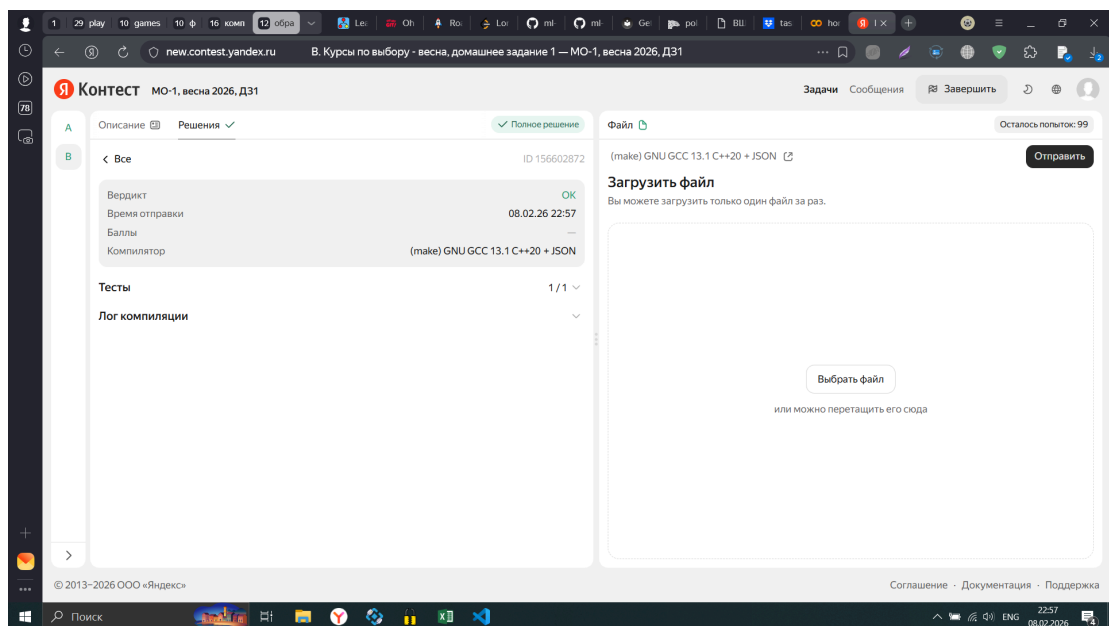
```
Out[81]: shape: (418, 3)
```

ID	course1	course2
str	str	str
"b8ab1a4ad4926caca7b82f8d1d11b2..."	"Теория и практика онлайн-экспе..."	"_"
"40c3044ac2c56f3b576601f7052685..."	"Машинное обучение 2"	"_"
"15a368d1368e4fd9b6dc0f2bc9362f..."	"Язык программирования Go"	"_"
"e6b3b61c7e6a0dcdd69e96325d12b2..."	"Машинное обучение 2"	"_"
"274f10a994f43939375bdfb3b3ac23..."	"Генеративные модели в машинном..."	"_"
...	...	...
"3501992c6601498ed885bfb2deefe..."	"Математические основы нейросет..."	"Методы сжатия и передачи медиа..."
"48fa126659b29ec4bfcd13cbc83506..."	"Децентрализованные системы"	"_"
"edb64996c64421378537d941a77426..."	"Промышленное программирование ..."	"_"
"d41f00051219658f10ba5bd41aba05..."	"Генеративные модели в машинном..."	"_"
"8edbf92455432dd49c7966b801be12..."	"???"	"_"

```
In [82]: df_part.write_csv('res_spring.csv')
```

Отправьте свой файл res_spring.csv в [Я.контекст](#) и прикрепите/укажите ниже ваш никнейм и id успешной посылки.

На всякий случай просим вас сдать вместе с ноутбуком файл res_spring.csv в anytask, **туда же и продублируйте никнейм из контекста и id успешных посылок.**



Даниил Перевалов dvperevalov@edu.hse.ru ID 156602872

Вставьте картинку, описывающую ваш опыт выполнения этого задания: